Modeling Divergence Between Community-Perceived and Automatically Calculated MTG Commander Brackets

Henrique Bidarte Massuquetti Marco Antônio Chitolina da Silva

Abstract

Magic: The Gathering (MTG) is a collectible card game in which players build decks from a large card pool and play them under format-specific rules. Commander is a casual multiplayer MTG format where players often discuss deck power before sitting down to play. Bracket labels help with that discussion, but they are subjective; automated calculators give another estimate from the decklist alone. We study 12,135 public Commander decks from Archidekt, each with an Archidekt bracket y_{arch} and an EDHPowerLevel calculator bracket y_{calc} in $\{2, 3, 4\}$. We encode each deck as Bag of Cards (BC) and Deck Features (DF), train six classifiers per representation to predict y_{arch}, and evaluate them with 3 repeated 5-fold nested cross-validation. DF models outperform BC models for every algorithm. The best individual model is DF Gradient Boosting with macro-F1 0.6908 ± 0.0093, while the best hard-voting ensemble reaches 0.6941 ± 0.0096. Raw y_{arch} and y_{calc} agree on 60.9% of decks; the model closest to the calculator, DF Random Forest, reaches 69.3% agreement with y_{calc} without using it as a training target. Interpretability links bracket predictions to Game Changers, mass land denial, tutors, combos, and high-power card identities. Instead of treating either source as ground truth, we describe where the community label and calculator agree, where they separate, and which deck signals explain that gap.

1 Introduction

Magic: The Gathering (MTG) is a collectible card game built around decks assembled from a large pool of cards. Commander is a casual multiplayer MTG format in which each player brings a 100-card singleton deck led by a legendary creature. Because each deck is built by a different player and the format is social, players often discuss deck power before a game starts. That conversation matters: a table feels different when one deck is much faster, more consistent, or more disruptive than the others.

Commander Brackets provide a five-level vocabulary for this discussion. Bracket 1 denotes low-power casual decks, bracket 5 denotes competitive EDH decks, and brackets 2–4 cover the common middle range where most casual Commander negotiation happens. A player assigns a bracket to their own deck, using a mixture of rule constraints and self-assessment. Automated calculators offer a second signal: they parse the same decklist and return a bracket from fixed rules over card identities, Game Changers, tutors, combos, prices, and popularity. The two readings are related, but they are not identical. We focus on that disagreement.

We collect 12,135 public Commander decks from Archidekt, each carrying an Archidekt bracket y_{arch} and an EDHPowerLevel calculator bracket y_{calc}, both in $\{2, 3, 4\}$. We represent each deck in two complementary ways. Bag of Cards (BC) is sparse and preserves card identity, so it can learn associations with specific staples and combo pieces. Deck Features (DF) is dense and hand-engineered, so it captures aggregate structure such as mana curve, Game Changer counts, tutors, and combo density. We train six classifiers retained by spot-checking, covering tree-based, ensemble, boosting, probabilistic, linear, and margin-based inductive biases, and compare their out-of-fold predictions $\hat{y}_{\text{arch}}$ against y_{calc}.

Research question. *To what extent do Archidekt-assigned Commander brackets diverge from automatically calculated bracket estimates, and which deck characteristics explain the disagreement?*

Contributions.

1. A curated dataset of 12,135 high-visibility Commander decks with both y_{arch} and y_{calc} labels in $\{2, 3, 4\}$, released along with extraction code.
2. A direct measurement of the raw divergence between y_{arch} and y_{calc} on the modeling set, including signed distribution and structural correlates.

3. A comparison of two deck representations (BC and DF) across six classifiers under repeated nested cross-validation, with per-fold preprocessing and shared splits.

4. A voting analysis over the top 3, 5, and 7 models by macro-F1, plus a separate selection of the best BC and DF models.

5. A comparison between y_{arch}, $\hat{y}_{\text{arch}}$, and y_{calc}, followed by interpretability analyses for the best DF and BC models.

Outline. Section 2 introduces MTG, Commander, Commander Brackets, and automated calculators. Section 3 documents data extraction, filtering, labels, exploratory divergence, and deck representations. Section 4 describes the spot-checking pass. Section 5 reports nested cross-validation, voting, and model selection. Section 6 compares models with the calculator. Section 7 interprets the best DF and BC models. Section 8 concludes with discussion, limitations, and future work.

2 Background

We introduce only the game context needed to follow the data and experiments. Game terminology used later (Game Changers, tutors, combos, EDHREC rank, salt score, layouts, keywords) is defined in Appendix A. A walkthrough of how a turn of Commander is played appears in Appendix A.1.

2.1 Magic: The Gathering

MTG is a turn-based collectible card game first published in 1993 [5]. A player builds a deck from a pool of tens of thousands of unique cards, draws a hand, and casts spells using a resource called *mana*. Each card has a type — creature, instant, sorcery, artifact, enchantment, planeswalker, or land — and a converted mana cost (CMC), which describes how much mana is needed to play it. Lands are the primary source of mana, while some non-land cards can also produce, reduce, or modify mana.

The game has many formats. Each defines deck-construction rules, the legal pool of cards, and the number of players at the table. Our work is restricted to one format, Commander.

2.2 Commander

Commander, also known as Elder Dragon Highlander (EDH), is the format we study. A deck has exactly 100 cards. One card is the *commander*: a legendary creature that begins in a separate zone and can be cast throughout the game. The remaining 99 cards must respect the commander's color identity, which is the set of mana colors appearing in its cost and rules text. With the exception of basic lands, each card may appear at most once; this is the *singleton* rule.

A default Commander game has four players, each playing their own deck and starting with 40 life. Because each player brings a different deck, a table can contain a wide range of strategies, speeds, budgets, and expectations. That variation is part of the format, but it also makes power-level communication important.

2.3 Why power balance matters

A Commander table works best when the four decks land in a comparable range of power. The format is organized around local playgroups, informal norms, and pre-game discussion rather than a single competitive ladder. When one deck is noticeably faster, more consistent, or includes a few cards that end the game on their own, the experience for the other players changes. The format does not enforce balance through matchmaking; it leaves the players to negotiate expectations before the game starts.

The metrics that the community has converged on are not deck win rates, but rather signals tied to deck *structure*: how many *Game Changers* a deck plays, how many *tutors*, how many *two-card combos*, the mana curve, and the rough cost of the deck. These are the same signals the automated calculators count, and they are the signals our DF representation captures.

2.4 Commander Brackets (community label y_{arch})

The format's official committee published a five-level bracket system in October 2024 [4]. Bracket 1 is casual and thematic: the deck is built for relaxed play, theme, and table expression. Bracket 5 is competitive EDH: the deck is tuned for efficient wins against other optimized decks. Brackets 2–4 are the middle range of casual Commander, and they are the focus of this work because they are the most common labels in our data and because the endpoints represent different use cases.

The bracket rules provide constraints, not a full classifier. For example, the system sets limits on Game Changer cards and describes broad expectations for speed, consistency, and disruption. These rules still require a player to read their own deck and decide where it fits. Two players can assign different brackets to similar decks because they may evaluate consistency, combo density, local norms, and threat level differently. In our dataset, y_{arch} is the bracket the deck creator selected on Archidekt, the deck-building platform we scraped [1].

2.5 Automated calculators (calculator label y_{calc})

Several community tools take a decklist as input and return a bracket from a deterministic rule set. We use EDHPowerLevel [2]. The calculator counts cards in named lists (Game Changers, tutors, extra-turn effects, mass land denial), counts known combo pieces, and uses card price and EDHREC popularity [3]. The result is a decklist-level estimate: it can use card content and external card statistics, but it cannot use player intent, local playgroup norms, or how the deck is piloted. The bracket it returns is deterministic given the inputs, but the inputs change over time because card prices and EDHREC ranks move daily.

The two readings disagree. A deck the player called bracket 3 may receive a calculator bracket of 2 or 4, and the rate of disagreement is large enough to motivate this work. Neither label is the single correct answer: y_{arch} reflects the player's reading, while y_{calc} reflects one rule set. We study the misalignment rather than arbitrate it.

3 Dataset Construction

3.1 Data sources

The community labels come from **Archidekt**, a public deck-building platform that stores user-created decklists, view counts, and the user-assigned bracket. Archidekt exposes a paginated REST API that we used to collect raw deck payloads, including the full mainboard and metadata.

The calculator labels come from **EDHPowerLevel**, a browser-based calculator that takes a pasted decklist and returns a bracket assignment and a continuous power score. EDHPowerLevel does not expose a programmatic API. We automated the interface with a headless Chromium browser driven by Playwright, pasting each decklist and reading the rendered bracket from the DOM.

3.2 Extraction pipeline

The extraction pipeline retrieved 14,421 deck records from the Archidekt API across the brackets we queried. We removed 1,467 duplicate deck IDs (the same deck listed under multiple paginated queries) and 4 deck records that produced identical card-list fingerprints, leaving 12,950 unique decks. Each remaining deck was then sent to EDHPowerLevel to obtain y_{calc} and the auxiliary power score.

By construction of the API query, every retained Archidekt deck satisfied:

- Public deck on Archidekt, format Commander;
- User-assigned bracket $y_{\text{arch}} \in \{2, 3, 4\}$;
- At least 1,000 views on Archidekt;
- Exactly 100 cards in the mainboard;
- Mainboard only — maybeboard, sideboard, tokens, and extras were ignored.

After joining y_{calc}, 815 decks (6.3%) received $y_{\text{calc}} \in \{1, 5\}$ and were dropped from the supervised setup, since brackets 1 and 5 are sparsely populated extremes. They are also qualitatively different from the middle of the scale: bracket 1 is reserved for very casual or thematic decks, while bracket 5 corresponds to competitive EDH, so including them would make the classifier spend capacity on edge cases rather

than on the common Commander power range where most community negotiation happens. This leaves the **modeling set of 12,135 decks**, with both labels in $\{2, 3, 4\}$. The 815 dropped decks are still useful as context for the divergence (Section 3.4).

3.3 Labels

Each deck has two labels: y_{arch}, the bracket the deck creator assigned on Archidekt, and y_{calc}, the bracket EDHPowerLevel returned for the same card list. We never treat either as ground truth: y_{arch} is subjective and varies across players, while y_{calc} reflects one rule set fed by time-varying inputs (prices, popularity). The supervised models are trained only on y_{arch}; y_{calc} is joined back after training for descriptive comparison.

3.4 Exploratory analysis of the divergence

Before fitting any model, we measure how often the two labels disagree, by how much, and in which direction. Table 1 is the central descriptive table for the dataset: it shows both the joint distribution of y_{arch} and y_{calc} and their marginal totals. Bracket 3 dominates both labels, but the off-diagonal cells show that the two label sources do not assign the same bracket to many decks.

Table 1: Joint counts $y_{\text{arch}} \times y_{\text{calc}}$ on the modeling set.

$y_{\text{arch}} \setminus y_{\text{calc}}$	2	3	4	Total
2	1,014	1,153	154	2,321
3	769	3,909	1,539	6,217
4	128	997	2,472	3,597
Total	1,911	6,059	4,165	12,135

The diagonal of Table 1 contains 7,395 decks, so $y_{\text{arch}} = y_{\text{calc}}$ for 60.9% of the modeling set. Allowing a one-bracket tolerance brings agreement to 97.7%; disagreements larger than one bracket occur in 282 decks (2.3%). The disagreement is also directional. With $\Delta = y_{\text{calc}} - y_{\text{arch}}$, the calculator assigns a higher bracket than Archidekt on 23.5% of decks and a lower bracket on 15.6%. The mean absolute difference is 0.414. In other words, when the two labels disagree, the calculator is more likely to move a deck upward than downward relative to the player label.

Only after this descriptive comparison do we move to supervised learning. We use y_{arch} as the target and treat y_{calc} as an external reference. To test whether deck power is better captured by specific card identity or by aggregate deck structure, we build two representations from the same filtered decks.

3.5 Bag of Cards (BC)

Each deck is encoded as a sparse vector indexed by the training-fold card vocabulary. The value on each dimension is the count of that card in the deck. Most non-basic cards appear at most once (the singleton rule); basic lands may appear multiple times.

The vocabulary is rebuilt inside each fold, from the training partition only. Cards never seen during training are dropped at test time. Cards appearing in fewer than 10 training decks are pruned (`min_df = 10`); this removes near-zero-variance columns that would inflate dimensionality without adding signal. The cutoff was chosen by sweeping $\{5, 10, 20\}$ during spot-checking (Section 4). The resulting matrix has about 11,000 columns.

BC keeps the card list visible to the model. Staples, combo pieces, and high-impact singletons all show up directly, so BC models learn associations between card identity and the community bracket.

3.6 Deck Features (DF)

Each deck is encoded as a dense vector of 114 aggregated structural attributes extracted from the Archidekt JSON. These features were handpicked to describe deck construction rather than author style: each feature belongs to a known Commander-relevant category such as mana curve, card type balance, combo density, price, popularity, or bracket-specific flags. The complete catalog, with the exact computation for each feature, is in Appendix C.

- **Mana base and curve.** Land count, basic / non-basic split, mana production per color, and CMC summary statistics plus per-bucket counts.
- **Card types.** Counts of creatures, instants, sorceries, artifacts, enchantments, planeswalkers, and non-land permanents.
- **Bracket-relevant flags.** Counts of Game Changers, tutors, extra-turn effects, and mass land denial cards — the four signals the bracket system uses to set hard caps.
- **Combo density.** Cards involved in atomic combos, in two-card combos, and total combo references.
- **Popularity and price.** EDHREC rank statistics, salt score statistics, total and per-card deck price.
- **Rarity.** Counts of common, uncommon, rare, and mythic cards.
- **Keywords.** Counts of evergreen keywords, total and distinct keyword counts.
- **Colors and identity.** Color count and per-color presence flags.
- **Multiface layouts.** Counts of transform, modal double-faced, split, and adventure cards.

We drop features derived from user-entered free text, notably Archidekt's per-card categories. An audit showed that 22.8% of decks use only user-defined labels that do not map to the standard functional vocabulary. Keeping those columns would mostly inject deck-authoring style rather than deck composition into the signal.

4 Spot-Checking

Nested cross-validation with hyperparameter search is the expensive end of the pipeline. We therefore begin with a low-cost spot-checking pass whose only goal is to remove algorithms that are clearly uncompetitive before the tuning stage.

4.1 Algorithm pool

We evaluate seven candidates chosen to span different inductive biases (Table 2). The pool is intentionally broad enough to include linear, tree-based, probabilistic, margin-based, and distance-based methods, while remaining feasible for both BC and DF.

Table 2: Candidate algorithms and their inductive biases.

Algorithm	sklearn class	Inductive bias
Decision Tree	`DecisionTreeClassifier`	Tree partitioning
Random Forest	`RandomForestClassifier`	Bagging
Gradient Boosting	`HistGradientBoostingClassifier`	Boosting
Naive Bayes	`MultinomialNB` (BC) / `GaussianNB` (DF)	Probabilistic
Logistic Regression	`LogisticRegression`	Linear parametric
Linear SVC	`LinearSVC`	Linear margin
K-Nearest Neighbors	`KNeighborsClassifier`	Distance-based

4.2 Hold-out protocol

Each candidate runs on both representations with scikit-learn 1.5 defaults (see Appendix E for the exact defaults). We use 5 stratified 80/20 hold-out splits with seeds $\{1, 2, 3, 4, 5\}$, stratified by y_{arch}, and report the mean and standard deviation of macro-F1 across seeds. Spot-checking is deliberately not a tuning pass: the hyperparameters are fixed at their defaults so that the ranking reflects the algorithm, not the search effort.

4.3 Results and selection

Table 3 lists the mean macro-F1 per (algorithm, representation). The two rankings agree on the bottom: KNN is last on both representations and shows the highest variance, so we drop it. Gradient Boosting, Logistic Regression, Linear SVC, Decision Tree, and Naive Bayes appear in both top-fives. Random Forest is the only asymmetric case: it makes the DF top-five (rank 2) but not the BC top-five (rank 6).

Table 3: Spot-checking macro-F1 (mean over 5 hold-out seeds) per algorithm and representation. ✓ marks membership in A_{union} used by nested CV.

Algorithm	BC mean	DF mean	In A_{union}
Gradient Boosting	0.6342	0.6850	✓
Random Forest	0.5243	0.6686	✓
Logistic Regression	0.5927	0.6672	✓
Linear SVC	0.5459	0.6225	✓
Decision Tree	0.5445	0.6042	✓
Naive Bayes	0.5570	0.5792	✓
KNN	0.4541	0.5241	—

We define $A_{\text{union}} = A_{\text{BC}} \cup A_{\text{DF}}$, where A_{BC} and A_{DF} are the top-five per representation. The union keeps Random Forest because the goal at this stage is to triage *algorithms*, not specific (algorithm, representation) pairs — under proper tuning, the asymmetric case may close. We carry the six algorithms in A_{union} forward into nested CV: Decision Tree, Random Forest, Gradient Boosting, Naive Bayes, Logistic Regression, Linear SVC. Total training surface: $6 \times 2 = 12$ models. Full table, standard deviations, and the `bc_min_df` sweep that fixed `min_df = 10` are in Appendix E.

5 Training and Model Selection

5.1 Nested cross-validation

We use nested cross-validation to keep hyperparameter selection separated from the final estimate of generalization:

- **Outer loop.** `StratifiedKFold` with 5 splits, repeated with 3 independent seeds (1, 2, 3), giving 15 outer evaluations per model.
- **Inner loop.** `StratifiedKFold` with 3 splits (seed = outer seed + 100), used as the grid-search loop. The inner loop picks the best configuration for each outer fold without ever touching the outer test set.

All 12 models share the same outer-fold indices. Fold indices are derived from the label vector and the seeds at runtime, so any machine with the same dataset and dependency versions produces the same splits. Because every model is evaluated on the same held-out rows, paired statistical tests across models are meaningful.

5.2 Per-fold preprocessing

All preprocessing is fit on the training partition of the current fold and applied to the held-out partition without ever seeing it. No statistic is computed on the full dataset before splitting, so test-fold information cannot leak into the model through imputation values, outlier caps, or scaling parameters. The full rationale — including why median imputation, why winsorize price, and why scaling is selective — is in Appendix B. The short version:

- **Imputation.** Missing EDHREC ranks and salt scores are filled with the training-fold median.
- **Winsorization.** `price_total` is clipped at the training-fold 99th percentile; the long tail (a handful of decks above $50,000) would otherwise dominate linear models and waste tree splits.
- **Scaling.** `StandardScaler` is applied to DF features for the scale-sensitive algorithms (Logistic Regression, Linear SVC, KNN). Tree-based models and Gaussian Naive Bayes receive unscaled DF features.

- **Leakage prevention.** The model sees only the current training partition. All fields derived from EDHPowerLevel (y_{calc}, the continuous power score, and any column prefixed `epl_`) are stripped from every feature matrix, in every fold. The only training target is y_{arch}; y_{calc} is joined back after training, only for descriptive comparison.

5.3 Hyperparameter grids

Each algorithm uses a compact grid of 24 configurations, chosen to cover the most impactful hyperparameters without exploding the search cost (Table 4). Inner-loop selection uses macro-F1.

Table 4: Hyperparameter grids for nested CV (24 configurations per algorithm).

Algorithm	Parameter	Values
Decision Tree	`max_depth`	{None, 10, 20}
	`min_samples_leaf`	{1, 5}
	`ccp_alpha`	{0, 0.005}
	`class_weight`	{None, balanced}
Random Forest	`n_estimators`	{100, 250, 500}
	`max_features`	{`sqrt`, `log2`}
	`min_samples_leaf`	{1, 2}
	`class_weight`	{None, balanced}
Gradient Boosting	`max_iter`	{200, 500}
	`learning_rate`	{0.05, 0.1, 0.2}
	`max_leaf_nodes`	{15, 31}
	`class_weight`	{None, balanced}
Naive Bayes (BC)	`alpha`	12 log-spaced values in $[10^{-3}, 100]$
	`fit_prior`	{True, False}
Naive Bayes (DF)	`var_smoothing`	24 log-spaced values in $[10^{-12}, 10^{-3}]$
Logistic Regression	`C`	{0.001, 0.1, 1, 100}
	`class_weight`	{None, balanced}
	`l1_ratio`	{0, 0.5, 1} (ElasticNet penalty)
Linear SVC	`C`	{0.001, 0.01, 0.1, 1, 10, 100}
	`class_weight`	{None, balanced}
	`penalty`	{L1, L2}

5.4 Evaluation metrics

Macro-F1 is the primary metric. It averages per-class F1 with equal weight, which is appropriate here: $y_{\text{arch}} = 3$ is 51.2% of the modeling set, and accuracy alone would mask poor recall on the minority classes. We also report accuracy, macro-precision, macro-recall, and per-class confusion matrices, all as mean $\pm$ standard deviation across the 15 outer folds.

5.5 Results

All 12 models completed the full 15 outer folds. Tables 5 and 6 report the nested-CV results separately for DF and BC, sorted by mean macro-F1 within each representation. Reading the tables separately is important: the project compares representations, but model selection for interpretability later needs the best model inside each representation.

The strongest individual model is DF Gradient Boosting with macro-F1 0.6908 ± 0.0093. The strongest BC model is BC Gradient Boosting with macro-F1 0.6433 ± 0.0121. DF is stronger than BC for every matched algorithm in the tuned comparison. The DF–BC macro-F1 gap ranges from $+0.0370$ for Naive Bayes to $+0.1251$ for Decision Tree. In this dataset, the dense handpicked representation gives every algorithm a more useful signal than sparse card identity alone.

The statistical tests agree with this ordering. A Friedman test over the 12 models and 15 paired folds rejects equal ranks (statistic = 158.84, $p < 10^{-6}$). The post-hoc Nemenyi critical difference at $\alpha = 0.05$ is 4.3025 ranks. DF Gradient Boosting has mean rank 1.07 and is significantly above all BC models; the

Table 5: Nested-CV results for DF models, sorted by macro-F1. Values are mean ± std over 15 outer folds.

Model	Macro-F1	Accuracy
`df_gradient_boosting`	0.6908 ± 0.0093	0.6978
`df_random_forest`	0.6733 ± 0.0076	0.7034
`df_decision_tree`	0.6727 ± 0.0100	0.6876
`df_logistic_regression`	0.6707 ± 0.0110	0.6944
`df_linear_svc`	0.6557 ± 0.0085	0.6584
`df_naive_bayes`	0.5905 ± 0.0093	0.5874

Table 6: Nested-CV results for BC models, sorted by macro-F1. Values are mean ± std over 15 outer folds.

Model	Macro-F1	Accuracy
`bc_gradient_boosting`	0.6433 ± 0.0121	0.6442
`bc_random_forest`	0.6326 ± 0.0122	0.6534
`bc_logistic_regression`	0.6236 ± 0.0096	0.6212
`bc_linear_svc`	0.6147 ± 0.0118	0.6281
`bc_naive_bayes`	0.5535 ± 0.0090	0.5636
`bc_decision_tree`	0.5475 ± 0.0116	0.5540

only non-significant Wilcoxon comparisons among top DF models are the near-ties among DF Decision Tree, DF Random Forest, and DF Logistic Regression.

As a leakage check, we also ran a 5-fold `GroupKFold` grouped by commander identity. The best models remain stable: DF Gradient Boosting scores 0.7021 under grouped evaluation, compared with 0.6908 in the stratified nested CV, and BC Gradient Boosting scores 0.6496 versus 0.6433. This suggests that the leading models are not simply memorizing commander-specific patterns.

5.6 Voting ensembles

We evaluate three simple hard-voting ensembles over the already-saved out-of-fold predictions: the top-3, top-5, and top-7 individual models ranked globally by mean macro-F1. No base model is retrained. Ties are resolved by the macro-F1 of the members voting for each class, with the smaller bracket as a final deterministic fallback. Table 7 lists both the members and the scores, so the reader can see how each ensemble expands the previous one.

Table 7: Hard-voting ensembles built from OOF predictions.

Ensemble	Members	Macro-F1	Δ vs best
`voting_top3`	DF Gradient Boosting; DF Random Forest; DF Decision Tree	0.6941 ± 0.0096	+0.0033
`voting_top5`	`voting_top3` plus DF Logistic Regression and DF Linear SVC	0.6932 ± 0.0076	+0.0024
`voting_top7`	`voting_top5` plus BC Gradient Boosting and BC Random Forest	0.6936 ± 0.0085	+0.0028

The best ensemble, `voting_top3`, improves over the best individual model by only +0.0033 macro-F1. The gain is measurable but small, which suggests that the strongest DF models already capture most of the learnable signal in y_{arch}. Adding the fourth and fifth DF models, and then the two strongest BC models, does not improve over the top-3 vote.

5.7 Selected models

After training, we keep three choices separate, summarized in Table 8. Based on Tables 5 and 6, the best individual model is DF Gradient Boosting and the best BC model is BC Gradient Boosting. Based on Table 7, the best ensemble is `voting_top3`. For interpretability, however, we select the best individual model separately by representation. We do not interpret the ensemble because it mixes model mechanisms and does not expose a single feature space.

Both selected Gradient Boosting models use the same modal configuration across their 15 outer folds: balanced class weights, learning rate 0.05, 200 boosting iterations, and 31 leaf nodes.

Table 8: Model selections used after training.

Role	Model	Repr.	Macro-F1
Best individual	`df_gradient_boosting`	DF	0.6908 ± 0.0093
Best ensemble	`voting_top3`	DF	0.6941 ± 0.0096
Best DF for interpretation	`df_gradient_boosting`	DF	0.6908 ± 0.0093
Best BC for interpretation	`bc_gradient_boosting`	BC	0.6433 ± 0.0121

6 Model-Calculator Comparison

We now compare the model predictions $\hat{y}_{\text{arch}}$ with y_{calc}. The goal is descriptive: no model is trained or retrained in this phase, and y_{calc} is never used as a target. The analysis reads the out-of-fold predictions saved by nested CV and voting, then compares them to the calculator label on the same deck rows.

Because the outer evaluation uses 3 repeats over the same 12,135 decks, the OOF comparison contains 36,405 rows. Each deck has a stable y_{calc}, but it contributes one prediction per repeat. This is why the OOF confusion matrices in the appendix sum to 36,405 rather than 12,135.

The comparison has two baselines. The first is the direct label comparison y_{arch} vs. y_{calc}, already shown as counts in Table 1. The second is the model comparison $\hat{y}_{\text{arch}}$ vs. y_{calc}. We report exact agreement, within-one-bracket agreement, mean absolute difference, and macro-F1 against y_{calc}. Exact agreement answers "how often does this source match the calculator?" Macro-F1 against y_{calc} answers the same question with class balance. The gap column is different: it is $|\text{macro-F1}(y_{\text{arch}}) - \text{exact agreement}(\hat{y}_{\text{arch}}, y_{\text{calc}})|$, so it measures how far a model's performance on the Archidekt target is from its raw agreement with the calculator. A high gap means the model can be good at predicting y_{arch} while not being the closest model to y_{calc}.

Table 9: Calculator-alignment highlights. The raw-label row compares y_{arch} directly with y_{calc}; all other rows compare model predictions $\hat{y}_{\text{arch}}$ with y_{calc}.

Case	Source	Exact	±1	Mean $\lvert\Delta\rvert$	F1 calc	F1 arch	Gap
Raw labels	y_{arch}	60.9%	97.7%	0.414	0.5843	—	—
Max agreement	DF RF	69.3%	99.5%	0.312	0.6674	0.6733	0.0194
Min agreement	BC DT	54.2%	96.6%	0.492	0.5328	0.5475	0.0057
Max gap	DF GB	64.0%	98.1%	0.379	0.6323	0.6908	0.0508
Min gap	BC NB	55.5%	97.1%	0.474	0.5462	0.5535	0.0010
Best BC	BC GB	60.3%	97.3%	0.424	0.5984	0.6433	0.0402
Best ensemble	Top-3 vote	66.7%	98.7%	0.346	0.6499	0.6941	0.0276

In the Source column of Table 9, GB denotes Gradient Boosting, RF denotes Random Forest, DT denotes Decision Tree, and NB denotes Naive Bayes.

Table 9 separates three interpretations that are easy to mix up. First, the raw labels agree with the calculator on 60.9% of OOF rows; this is the baseline disagreement between Archidekt and EDHPowerLevel. Second, some models are closer to the calculator than the raw labels are. DF Random Forest reaches 69.3% exact agreement and macro-F1 0.6674 against y_{calc}, although it was trained only on y_{arch}. Third, the best model for y_{arch} is not the closest model to the calculator. DF Gradient Boosting has the highest individual macro-F1 on y_{arch}, but only 64.0% exact agreement with y_{calc}, giving the largest target/calculator gap in the table.

The main takeaway is that calculator alignment is related to, but not identical to, predictive performance on the Archidekt target. Models that use DF can learn structure shared by the community label and the calculator, especially because both are sensitive to deck-level signals such as Game Changers and tutors. At the same time, a model optimized for y_{arch} can deviate from y_{calc}, which is expected if player labels encode social or contextual readings that the calculator does not capture.

7 Interpretability

Interpretability uses exactly the best individual model from each representation, selected in Section 5.7: DF Gradient Boosting and BC Gradient Boosting. This separation matters. The best global model is a DF model and the best ensemble mixes multiple estimators, but the paper needs to explain both representations. We therefore ask two representation-specific questions. For DF, which handpicked deck

attributes does the best structured model depend on? For BC, which card identities are enriched in each predicted bracket?

The performance estimates remain those from nested CV. For interpretation only, each selected model is fit again on the full modeling set or on an interpretability split, depending on the method. These final fits are not used to claim new predictive performance.

7.1 Deck Features

For DF, we compute permutation importance on a stratified 20% hold-out with 10 repeats. HistGradientBoosting does not expose impurity-based feature importances, and permutation importance measures the drop in macro-F1 after shuffling each feature. A large value means the model loses predictive performance when that feature is disconnected from the label. Table 10 reports the top features.

Table 10: Top DF permutation importances for `df_gradient_boosting`.

Feature	PI mean	PI std
`game_changer_count`	0.2281	0.0096
`mass_land_denial_count`	0.0288	0.0029
`unique_atomic_combo_refs_count`	0.0143	0.0052
`tutor_count`	0.0097	0.0038
`cmc_1_count`	0.0060	0.0010
`land_count`	0.0043	0.0019

The feature ranking is concentrated. `game_changer_count` has permutation importance 0.2281, while the second feature, `mass_land_denial_count`, has importance 0.0288. The model is not using only one feature, but the strongest split between brackets is tied to a signal explicitly used by the bracket system. The direction is also consistent with the bracket definition. In the validation split, mean `game_changer_count` is 0.001 for decks predicted as bracket 2, 1.499 for bracket 3, and 4.146 for bracket 4. The next features refine the same notion of power: mass land denial captures resource denial, combo references capture deterministic wins, and tutor count captures consistency.

The same pattern appears in the calculator comparison. Rows where $\hat{y}_{\text{arch}} = y_{\text{calc}}$ have higher `game_changer_count` than divergent rows (2.165 vs. 0.939). When clear bracket-defining signals are present, the model and calculator align more often. Divergence is therefore concentrated in decks where power depends on context, synergy, or player interpretation rather than on obvious bracket flags.

7.2 Bag of Cards

For BC, dense permutation importance over more than 11k card features would require repeatedly shuffling a very large sparse matrix. We therefore analyze card lift over OOF predictions:

$$\text{lift}(k, c) = \frac{P(c \text{ present} \mid \hat{y}_{\text{arch}} = k)}{P(c \text{ present})}.$$

Cards with fewer than 10 unique decks are filtered. Lift asks whether a card is more common inside a predicted bracket than in the dataset overall. A lift of 3 means the card appears about three times as often in decks predicted as that bracket. Table 11 shows the strongest bracket 4 lifts.

Table 11: Top bracket 4 card lifts for `bc_gradient_boosting`.

Card	Lift	Freq. in predicted bracket 4
Blood Moon	3.325	3.5%
Winter Moon	3.325	2.1%
Ruination	3.325	0.5%
Harbinger of the Seas	3.278	1.3%
Back to Basics	3.186	0.8%
Chrome Mox	3.185	6.7%
Imperial Seal	3.160	2.1%
Grim Monolith	3.138	2.1%

The bracket 4 list is dominated by cards associated with high-power Commander: fast mana, tutors, and resource denial. Several of the top cards are not merely efficient; they change how the table is allowed to play. *Blood Moon*, *Winter Moon*, *Ruination*, and *Back to Basics* attack mana bases. *Chrome Mox*, *Grim Monolith*, and similar cards accelerate early turns. *Imperial Seal* increases consistency by finding a specific card. This is a card-identity version of the same signal seen in DF: high brackets are associated with faster, more consistent, and more restrictive deck construction.

The divergence lift analysis separates concordant and divergent OOF rows. Cards enriched in concordant rows include *Blood Moon*, *Imperial Seal*, *Force of Will*, and *Chrome Mox*; these are cards for which the model and calculator tend to agree on high power. Rows where the model predicts below the calculator ($\hat{y}_{\text{arch}} < y_{\text{calc}}$) are enriched for cards such as *Vorinclex, Voice of Hunger*, *Expropriate*, *Time Stretch*, and *Time Sieve*. These cards can be strong, but their table impact depends on the surrounding deck and playgroup. This explains why a deterministic calculator and a model trained on community labels can diverge: they react to overlapping signals, but they do not encode the same notion of context.

8 Conclusion

We built a reproducible pipeline for studying the divergence between community-assigned and automatically calculated Commander brackets. On 12,135 Archidekt decks with both labels in brackets 2–4, raw agreement between y_{arch} and y_{calc} is 60.9%, with the calculator assigning higher brackets more often than players do. Across six algorithms and two representations, DF models outperform BC models under nested cross-validation; the best individual model is DF Gradient Boosting with macro-F1 0.6908 ± 0.0093, and the best voting ensemble improves this only slightly to 0.6941.

Interpretability points to a coherent explanation: aggregate deck structure, especially Game Changer count, mass land denial, combo references, and tutors, dominates the DF model, while BC lift recovers recognizable high-power cards such as *Blood Moon*, *Chrome Mox*, *Imperial Seal*, and *Mana Vault*. The models therefore do not decide whether the community or the calculator is "right"; they expose where their readings overlap and where they diverge.

8.1 Discussion, Limitations, and Future Work

Taken together, the results show that structured deck features are more predictive of the Archidekt bracket than raw card identity in this setting. DF wins for every tuned algorithm, and the best DF model also has the best statistical rank. This does not mean that individual cards are irrelevant: the BC lift analysis recovers familiar high-power cards, especially in bracket 4. Rather, the nested-CV comparison suggests that Archidekt brackets are better approximated by aggregate structure — Game Changers, tutors, land denial, combo density, curve, and mana base — than by sparse card identity alone.

The comparison with y_{calc} is descriptive. We do not claim that the calculator is a ground truth label, nor that y_{arch} is one. DF Random Forest agrees with y_{calc} more often than the raw Archidekt labels do, which shows that part of the learned signal overlaps with the calculator's rules. Conversely, the best predictor of y_{arch}, DF Gradient Boosting, is not the model most aligned with the calculator. This gap identifies where a model captures community-specific structure rather than reproducing the automated bracket logic.

Several limitations remain. First, y_{calc} is a snapshot. EDHPowerLevel uses time-varying signals such as prices and EDHREC ranks, and a small re-query audit found that 3 of 90 decks changed brackets several weeks later. Second, the sample contains public, high-visibility Archidekt decks; private decks and low-view experimental lists may behave differently. Third, the calculator encodes one specific interpretation of Commander power. A different calculator might disagree with EDHPowerLevel in ways that mirror the community-calculator gap studied here. Finally, bracket prediction is not win-rate prediction. The models learn labels attached to decklists, not the strength, play experience, or social acceptability of a deck at a table.

Future work should test temporal stability by re-querying calculator labels over time, include additional deck-building platforms, and evaluate whether ordinal classifiers or calibrated models better match the ordered nature of brackets. A separate design is needed for brackets 1 and 5, since they represent endpoints rather than the common Commander power range modeled here.

References

[1] Archidekt – MTG deck builder. https://archidekt.com. Accessed: 2026-05-23.

[2] EDHPowerLevel – commander power level calculator. https://www.edhpowerlevel.com. Accessed: 2026-05-23.

[3] EDHREC – commander recommendations and statistics. https://edhrec.com. Accessed: 2026-05-23.

[4] Commander Rules Committee. Commander Brackets. https://mtgcommander.net/index.php/2024/10/17/october-17-2024-announcement/, 2024. Accessed: 2026-05-23.

[5] Wizards of the Coast. Magic: The Gathering. https://magic.wizards.com, 1993. Accessed: 2026-05-23.

The appendices collect supplementary material. Appendix A provides MTG and Commander background, including a turn-by-turn walkthrough of how a Commander game is played. Appendix B expands the preprocessing rationale. Appendix C lists every Deck Features attribute and how it is computed. Appendix D adds dataset diagnostics. Appendix E gives the full spot-checking results. Appendix F summarizes model verification and statistical tests. Appendix G details voting and selected models. Appendix H adds calculator-comparison details. Appendix I expands interpretability. Appendix J gives reproducibility and AI-use statements.

A MTG Terminology and Gameplay

For readers who want more game context, this appendix collects the *Magic: The Gathering* (MTG) concepts referenced throughout the paper. Readers already familiar with the game may safely skip it; the main text is self-contained for the methodology and results. In-text references point at the specific subsection where each concept is defined.

A.1 How a turn of Commander is played

A Commander game runs as a sequence of turns. With four players, the table cycles clockwise from a starting player chosen randomly. Each player starts with 40 life and a 7-card hand drawn from their 100-card library, plus their commander placed in the command zone outside the deck.

A turn is divided into phases. The active player goes through them in order:

1. **Beginning phase.** Untap previously tapped permanents; check upkeep triggers; draw one card (the player on turn one skips the draw).
2. **First main phase.** Cast spells and play at most one land. Lands tap to produce mana, which is spent on the same turn to pay spell costs. The commander can be cast from the command zone here. Casting the commander again after it has died costs an extra {2} mana per prior cast (the *commander tax*).
3. **Combat phase.** Declare attackers; each attacker chooses one defending player. The defender declares blockers. Damage resolves; creatures that take damage equal to or above their toughness go to the graveyard. A player who takes 21 combat damage from a single commander over the course of the game also loses, independent of life total.
4. **Second main phase.** Another opportunity to cast spells and (if not already done) play a land.
5. **End phase.** Resolve end-of-turn triggers, discard down to seven cards, pass to the next player.

A player loses when their life total drops to zero, when they would draw from an empty library, when they accumulate 10 poison counters, or when they take 21 damage from a single commander. The last player remaining wins. Games typically last 30–90 minutes.

The format's casual nature means *social* interactions also shape the table: house rules, threat assessment, table talk, and the rate at which any one player commits resources all matter, but they are not visible in a decklist. The brackets system was designed to summarize this informally before the first turn is played.

A.2 Game structure

Commander and command zone. The *commander* is a *legendary creature* that begins the game in a separate zone, the *command zone*, and may be cast from there at any point during the game. The commander's *color identity* — the set of mana colors appearing in its cost and rules text — constrains which cards the other 99 cards of the deck may include.

Singleton format. Commander is a *singleton* format: each deck may contain at most one copy of any non-basic land card. *Basic lands* — generic resource-producing cards with no unique rules text — may be included in any quantity.

Player count. Commander is typically played in pods of four, though the format admits any number of players from two upward. The four-player pod is the configuration around which the bracket discourse is framed.

A.3 Cards, mana, and the curve

Mana and mana cost. *Mana* is the resource in MTG, produced mostly by *land* cards. Players play at most one land per turn and spend mana to pay the *mana cost* of spells. The *converted mana cost* (CMC), also called *mana value*, is the total amount of mana required to cast a card, regardless of color.

Mana curve. The distribution of CMC values across a deck is the *mana curve*. Cards with CMC 0–1 are cheap and fast; cards with CMC 5+ are expensive and slow. The shape of the curve reflects the deck's expected speed and consistency.

Card types. MTG defines several primary *card types*: *creatures* attack and block; *instants* and *sorceries* are one-time spells; *artifacts* are objects with persistent effects; *enchantments* are ongoing magical effects; *planeswalkers* are recurring-ability allies; *lands* produce mana. Cards that remain on the battlefield after being played are collectively called *permanents*.

Rarity tiers. MTG cards are printed at four rarity levels that broadly correlate with power and availability: *common* (most frequent), *uncommon*, *rare*, and *mythic rare* (least frequent). A deck's rarity distribution is a rough proxy for power ceiling and price.

Multiface layouts. Some MTG cards have multiple faces. *Transform* cards flip between two faces under specific conditions. *Modal double-faced cards* let the player choose which face to play at cast time. *Split cards* contain two independent spells on one card. *Adventure* cards bundle a creature with an instant or sorcery.

Mana colors. MTG uses five colors of mana: *White* (W), *Blue* (U), *Black* (B), *Red* (R), and *Green* (G), each associated with a distinct strategic identity.

A.4 Power indicators

Staples. A *staple* is an informal term for a card so generically efficient that it appears in a large fraction of competitive or semi-competitive decks regardless of strategy. Examples: *Sol Ring* (a cheap mana-producing artifact), *Command Tower* (a land that produces any color in the commander's identity).

Game Changers. *Game Changers* are cards officially designated by the Commander Rules Committee as capable of ending games or dominating tables on their own. Examples: *Thassa's Oracle*, *Rhystic Study*. Their count in a deck is the primary signal the bracket system uses to distinguish brackets 3 and 4.

Tutors. A *tutor* is a card that searches the player's library for another specific card and places it in the player's hand or in play, raising deck consistency. Examples: *Demonic Tutor*, *Enlightened Tutor*.

Combos. A *combo* is a combination of two or more cards that, once assembled, produces a game-winning or overwhelming effect (infinite damage, infinite mana, deck-out). An *atomic combo* is one in which every piece is necessary and sufficient; a *two-card combo* is a particularly efficient variant requiring only two pieces.

Extra-turn effects. Cards that grant a player additional turns beyond the standard cycle (*Time Warp*, *Temporal Manipulation*) create significant tempo advantages and are considered high-impact.

Mass land denial. Cards that destroy or otherwise neutralize all players' lands simultaneously (*Armageddon*, *Ravages of War*) are socially contentious in Commander because they can render the game unplayable for the rest of the table.

A.5 Keywords

Keywords are shorthand labels for recurring rules text. *Evergreen* keywords appear across many sets and are always available; examples include *Flying* (can be blocked only by creatures with Flying or Reach), *Trample* (excess combat damage carries over to the defending player), *Haste* (can attack the turn it enters play), *Hexproof* (cannot be targeted by opponents), and *Deathtouch* (destroys any creature it damages).

A.6 Community resources

EDHREC rank. EDHREC (`https://edhrec.com`) aggregates published Commander decklists and computes per-card popularity statistics. The *EDHREC rank* orders cards by inclusion frequency: rank 1 is the most widely played card, with higher numbers denoting less popular cards.

Salt score. The *salt score* is a community-sourced metric compiled by EDHREC that measures how much frustration a card is reported to cause opposing players. High-salt cards typically lock opponents out, induce repeated random effects, or end games in ways perceived as unfun.

B Preprocessing — Extended Rationale

Section 5.2 states the four transformations applied during nested CV. Here we give the rationale behind each choice.

Per-fold fitting. Every statistic used by preprocessing — the median for imputation, the percentile for winsorization, the mean and standard deviation for scaling — is computed from the training partition of the current outer fold only. The model is never shown anything derived from the held-out partition of the same fold or from any other fold. This is a hard requirement of the protocol: a model that "learned" a winsorization cap from the test fold would not be measuring generalization, it would be measuring how well the test fold fits its own statistics.

Imputation. The EDHREC rank and salt score features are produced by an external community service and are missing for rare or recently released cards that EDHREC has not yet indexed. The affected rate is small. We fill missing values with the training-fold median rather than fitting a more elaborate imputer because the missingness is a property of the card, not of the deck. Median is preferred over mean because both fields have heavy-tailed distributions (a few cards have very high ranks or salt scores), and over zero because zero is not a neutral value on either scale — EDHREC rank 0 would mean the single most popular card; salt 0 would mean a non-controversial card.

Winsorization. Total deck price (`price_total`) is clipped at the 99th percentile of the training fold. Most decks cost a few hundred USD, but a small tail of collector-grade decks exceeds $50,000, driven by a handful of reserved-list cards. Without clipping, scale-sensitive linear models would put disproportionate weight on those outliers, and tree splits would land on degenerate thresholds determined by a single deck. Winsorization keeps the signal in the model while removing the long-tail leverage. We prefer winsorization to a log transform because the unit of the feature stays "dollars," which keeps downstream interpretability of coefficients and feature importances straightforward.

Scaling. `StandardScaler` centers each DF feature to zero mean and unit variance, fit on the training fold. We apply it to Logistic Regression, Linear SVC, and KNN, whose objective functions or distance metrics are scale-dependent and would otherwise be dominated by high-magnitude features such as `price_total` or `edhrec_rank_mean`. Tree-based models (Decision Tree, Random Forest, Gradient Boosting) and Gaussian Naive Bayes are invariant to monotonic per-feature rescaling, so we leave them unscaled to avoid an unnecessary transform that would obscure feature-importance interpretation.

Leakage prevention. The supervised target is y_{arch}. Every field that the EDHPowerLevel pass produced — y_{calc}, the continuous power score, efficiency, tipping point, and any column prefixed `epl_` — is stripped from the feature matrix before fitting. y_{calc} is only joined back during the descriptive comparison after training. Without this rule, a trivial classifier that simply reproduced the EDHPowerLevel bracket would score well while explaining nothing about the Archidekt label.

C Deck Features (DF) Catalog

This appendix lists every feature in the Deck Features representation introduced in Section 3.6, grouped by the same families. Each feature is computed strictly from data observable in the Archidekt raw payload (mainboard rows, oracle card metadata, and printing-level rarity/price), so no external state is queried at training or inference time. Per-card quantities are respected throughout: a feature defined as a count multiplies the per-card boolean by the row's `quantity` (relevant for basic lands and a handful of cards that may legally appear in multiples).

C.1 Basic structure

- `unique_card_count`: number of distinct `oracle_uid`s in the mainboard.
- `non_commander_card_count`: total quantity of mainboard cards that are not the commander.
- `commander_card_count`: total quantity of commander cards (1 for single-commander decks, 2 for Partner pairs and Background combinations).
- `has_companion`: boolean flag indicating whether a Companion card is registered for the deck. Always false in the modeling set because Companions are excluded upstream.

C.2 Colors and color identity

- `deck_color_count`: cardinality of the union of `colorIdentity` across all mainboard cards, in $\{0, \ldots, 5\}$.
- `has_W`, `has_U`, `has_B`, `has_R`, `has_G`: boolean flags, one per color, true if any card in the mainboard has that color in its identity.

C.3 Mana base

- `land_count`, `nonland_count`: total quantities of lands and non-lands in the mainboard.
- `basic_land_count`, `nonbasic_land_count`: split of the land count between basic lands (the five color basics plus Wastes, or any card with `superType=Basic` and `type=Land`) and non-basics.
- `land_mana_production_W`, `_U`, `_B`, `_R`, `_G`, `_C`: for each color (including colorless), the count of mainboard lands whose `manaProduction` entry includes that color.
- `lands_that_produce_multiple_colors_count`: count of lands that produce more than one colored mana (colorless not counted).
- `lands_that_produce_colorless_count`: count of lands whose `manaProduction` includes the colorless "C" symbol.

C.4 Mana curve (CMC)

CMC is taken from the canonical `oracleCard.cmc` field. Archidekt's user-overridable `custom_cmc` is deliberately ignored: an audit showed that about 19% of decks use it on at least one row, mixing legitimate cases (X-spells) with subjective preference, so using it would inject author-specific noise into a feature that is meant to describe the card.

- `cmc_mean`, `cmc_median`, `cmc_std`: mean, median, and population standard deviation of CMC across all mainboard cards (lands counted at CMC 0).
- `nonland_cmc_mean`, `nonland_cmc_median`, `nonland_cmc_std`: same statistics restricted to non-land cards.
- `cmc_0_count`, `cmc_1_count`, ..., `cmc_5_count`, `cmc_6_plus_count`: histogram of non-land cards by integer CMC bucket, with CMCs of 6 or more collapsed into a single bucket.

C.5 Card types

- `creature_count`, `instant_count`, `sorcery_count`, `artifact_count`, `enchantment_count`, `planeswalker_count`: count of mainboard cards (respecting quantity) whose `types` array contains the corresponding type.
- `noncreature_spell_count`: count of mainboard cards that are neither creatures nor lands.
- `nonland_permanent_count`: count of mainboard cards that are non-land permanents (creature, artifact, enchantment, or planeswalker).

C.6 Bracket-relevant flags

These count cards flagged by external sources whose presence the bracket system uses to set hard caps (Appendix A.4).

- `game_changer_count`: count of cards with `oracleCard.gameChanger = true`.
- `extra_turns_count`: count of cards with `oracleCard.extraTurns = true`.
- `tutor_count`: count of cards with `oracleCard.tutor = true`.
- `mass_land_denial_count`: count of cards with `oracleCard.massLandDenial = true`.
- `two_card_combo_singleton_count`: count of cards flagged as a singleton piece of a known two-card combo by Archidekt's combo registry.

C.7 Combo density

- `cards_with_atomic_combos_count`, `cards_with_potential_combos_count`: number of mainboard cards that participate, respectively, in at least one atomic combo or at least one potential combo according to Archidekt's combo data.
- `atomic_combo_refs_total`, `potential_combo_refs_total`: total references summed across cards (a card that participates in three combos contributes 3).
- `unique_atomic_combo_refs_count`, `unique_potential_combo_refs_count`: number of distinct combo identifiers referenced by the deck.
- `two_card_combo_ids_total`: total references across cards to two-card combos.
- `unique_two_card_combo_ids_count`: number of distinct two-card combo identifiers referenced by the deck.

C.8 Popularity, salt, and price

- `edhrec_rank_mean`, `edhrec_rank_median`, `edhrec_rank_std`: aggregate statistics of EDHREC rank across mainboard cards. Cards with missing EDHREC rank are excluded from these statistics; the resulting missing value is imputed downstream by the fold-fitted median (Appendix B).
- `salt_mean`, `salt_median`, `salt_std`: aggregate statistics of EDHREC salt score, treated analogously.
- `price_total`, `price_mean`, `price_median`, `price_std`: aggregate statistics of paper non-foil price in USD. Vendor preference is TCGPlayer market, falling through Card Kingdom, CardMarket, Star City Games, MagicPlayer, and CardTrader. `price_total` is winsorized at the training-fold 99th percentile before modeling.

C.9 Rarity

- `common_count`, `uncommon_count`, `rare_count`, `mythic_count`: counts of mainboard cards by printing-level rarity, taken from the specific printing the deck uses.

C.10 Keywords

Keywords are extracted from `oracleCard.keywords`.

- `keyword_total_count`: total keyword occurrences across the mainboard (a card with two keywords contributes 2).
- `distinct_keyword_count`: number of distinct keyword strings observed in the mainboard.
- `flying_count`, `trample_count`, `haste_count`, `hexproof_count`, `ward_count`, `indestructible_count`, `lifelink_count`, `menace_count`, `vigilance_count`, `deathtouch_count`, `flash_count`, `equip_count`, `annihilator_count`, `cascade_count`: per-keyword card counts for the tracked evergreen and high-impact keywords.
- `protection_keyword_count`: count of cards whose keyword list contains any `Protection from ...` variant.

C.11 Super- and subtypes

- `legendary_count`, `snow_count`: counts of cards with the corresponding supertype.
- `distinct_subtype_count`: number of distinct subtypes observed across the mainboard.
- `most_common_subtype_count`: total count of cards belonging to the most frequent subtype in the deck (a tribal proxy).
- `equipment_subtype_count`, `aura_subtype_count`, `vehicle_subtype_count`: counts of cards with the corresponding subtype.

C.12 Multiface layouts

- `multiface_card_count`: count of cards that expose more than one face.
- `layout_normal_count`, `layout_transform_count`, `layout_modal_dfc_count`, `layout_split_count`, `layout_adventure_count`: per-layout card counts for the tracked layouts.
- `cards_with_faces_count`: count of cards whose `faces` array is non-empty.
- `total_face_count`: total number of faces summed across the deck.
- `max_faces_on_card`: maximum face count observed on any single card.

Features deliberately excluded. Several candidate features were considered and dropped during feature engineering, for transparency: Archidekt's per-card category strings (22.8% of decks use only user-defined labels that do not map to the standard functional vocabulary, so the resulting columns would be near-constant noise rather than structural signal); a generic `permanent_count` (its previous definition incorrectly excluded lands, making it identical to `nonland_permanent_count`); `rare_mythic_count` (the sum of two other columns, redundant for any linear model); `cmc_min`/`cmc_max` (`min` is almost always 0 due to basic lands, `max` is set by a single high-CMC card); `edhrec_rank_min` and `salt_max` (both dominated by a single staple); `price_max` (single expensive card); and per-card keyword count statistics (inflated by flavor and set-specific keywords).

D Dataset Diagnostics

We include two dataset diagnostics omitted from the main text for space. First, y_{calc} is treated as a collection-time snapshot. EDHPowerLevel uses time-varying signals such as prices and EDHREC rank. In a re-query audit of 90 decks several weeks after the original pass, 87 decks (96.7%) received the same bracket; the 3 changed decks were near a bracket boundary.

Second, raw disagreement between y_{arch} and y_{calc} varies with deck structure. Mean $|y_{\text{calc}} - y_{\text{arch}}|$ is largest when decks lack clear bracket-defining signals and decreases when Game Changer counts are high: decks with 0 Game Changers have mean absolute difference 0.568, while decks with 5 or more are almost always assigned the same bracket by both sources. Tutor count, price quintile, EDHREC rank, and color count show smaller trends in the same diagnostic reports.

E Spot-Checking — Full Results

This appendix gives the full spot-checking results, software versions, and default hyperparameters for Section 4.

Software versions. Python 3.11, `scikit-learn` 1.5, `scipy` 1.13, `numpy` 1.26, `pandas` 2.2. Hold-out splits use `StratifiedShuffleSplit` (80/20, 5 seeds) and are reproducible from the seed list $\{1, 2, 3, 4, 5\}$.

Default hyperparameters. Each candidate runs with its scikit-learn 1.5 defaults, except where the default is not viable on this scale. The notable defaults are:

- `DecisionTreeClassifier`: `criterion="gini"`, no depth limit.
- `RandomForestClassifier`: 100 trees, `max_features="sqrt"`, `n_jobs=-1`.
- `HistGradientBoostingClassifier`: `max_iter=100`, `learning_rate=0.1`, `max_leaf_nodes=31`.
- `MultinomialNB`: `alpha=1.0`, BC representation.
- `GaussianNB`: `var_smoothing=1e-9`, DF representation.
- `LogisticRegression`: `C=1.0`, `penalty="l2"`, `max_iter=1000`; solver `lbfgs` on DF and `saga` on BC.
- `LinearSVC`: `C=1.0`, `penalty="l2"`, `loss="squared_hinge"`, `max_iter=5000`.
- `KNeighborsClassifier`: `n_neighbors=5`, `weights="uniform"`.

`StandardScaler` is applied to DF for the scale-sensitive algorithms; tree-based models and `GaussianNB` receive unscaled features.

BC vocabulary threshold. `bc_min_df` was selected by sweeping $\{5, 10, 20\}$ on the BC representation. `min_df = 10` produced the highest mean macro-F1 across seeds and is fixed for all subsequent spot-checking and nested CV runs.

Table 12: Spot-checking macro-F1 (mean ± std over 5 stratified 80/20 hold-outs) per algorithm and representation, with the top-5-per-representation selection. ✓ marks membership in A_{union}.

Algorithm	BC mean	BC std	DF mean	DF std	In A_{union}
Gradient Boosting	0.6342	0.0064	0.6850	0.0049	✓
Logistic Regression	0.5927	0.0066	0.6672	0.0050	✓
Random Forest	0.5243	0.0119	0.6686	0.0103	✓
Linear SVC	0.5459	0.0107	0.6225	0.0112	✓
Decision Tree	0.5445	0.0041	0.6042	0.0102	✓
Naive Bayes	0.5570	0.0070	0.5792	0.0080	✓
KNN	0.4541	0.0177	0.5241	0.0113	—

Per-algorithm macro-F1. Table 12 reports mean (and standard deviation) of macro-F1 across the five hold-out seeds. The five highest-ranked algorithms per representation are kept; their union A_{union} defines the algorithm set carried forward to nested CV.

Naive Bayes and Random Forest are retained asymmetrically. Naive Bayes makes the top-five for BC (rank 3) but not for DF (rank 6); Random Forest makes the top-five for DF (rank 2) but not for BC (rank 6). The union rule keeps both: the goal of spot-checking is to triage algorithms, not (algorithm, representation) pairs — under proper tuning the asymmetric case may close, and we prefer to spend the tuning budget on an algorithm that already performs in one representation rather than rule it out from its default behavior in the other.

Spot-checking summary. The per-seed results follow the same pattern as Table 12: Gradient Boosting dominates on both sides, DF consistently outperforms BC, and KNN is the weakest candidate in both representations.

F Training Verification and Statistical Tests

This appendix summarizes the verification phase applied before voting and model selection. The verification phase is a quality gate: it checks that all expected models exist, all outer folds completed, and all models share the same fold identities before any paired comparison is reported.

Table 13: Model verification checks.

Check	Result
Expected individual models	12/12 present
Outer folds per model	15/15 for every model
Shared fold identities	Same repeat/fold IDs across all models
Inner-grid size	24 configurations per algorithm/fold
Best hyperparameters	Rank-1 inner macro-F1 recovered from CV results
Training target	y_{arch} only; y_{calc} excluded from features

To test whether models memorize commander-specific patterns, we also run a 5-fold `GroupKFold` grouped by commander identity. Table 14 reports the two models later used for interpretation. The grouped scores are close to the stratified nested-CV scores, which supports the conclusion that the leading models do not rely only on commander identity.

Table 14: Commander-grouped robustness check for selected models.

Model	Nested CV macro-F1	GroupKFold macro-F1	Group - nested
`df_gradient_boosting`	0.6908	0.7021	+0.0113
`bc_gradient_boosting`	0.6433	0.6496	+0.0063

The paired statistical tests use the same 15 outer scores for every model. A Friedman test rejects equal ranks across the 12 individual models (statistic = 158.84, $p < 10^{-6}$). The Nemenyi critical difference at $\alpha = 0.05$ is 4.3025 ranks. DF Gradient Boosting has mean rank 1.07 and is significantly above all BC

models. Wilcoxon tests show that the top DF models are close to each other, while the DF-vs-BC gap is consistent across algorithms.

G Voting and Model Selection Details

Voting uses saved OOF predictions and never retrains a base model. The member lists are fixed by global macro-F1 ranking of individual models. Hard-vote ties are resolved by the summed macro-F1 of the models voting for each class; if a tie remains, the smaller bracket is selected.

Table 15: Voting ensemble members.

Ensemble	Members
`voting_top3`	DF Gradient Boosting; DF Random Forest; DF Decision Tree
`voting_top5`	DF Gradient Boosting; DF Random Forest; DF Decision Tree; DF Logistic Regression; DF Linear SVC
`voting_top7`	DF Gradient Boosting; DF Random Forest; DF Decision Tree; DF Logistic Regression; DF Linear SVC; BC Gradient Boosting; BC Random Forest

Model selection for interpretability is representation-specific. The best DF model is DF Gradient Boosting; the best BC model is BC Gradient Boosting. This rule prevents the interpretability phase from interpreting only the best global model and omitting the best model for one representation.

H Calculator Comparison Details

The model-calculator comparison uses 36,405 OOF rows: 12,135 decks times 3 repeats. The calculator label is stable per deck. Table 16 repeats the raw y_{arch} vs. y_{calc} matrix on the OOF-expanded base used for direct comparison with model predictions.

Table 16: OOF-expanded $y_{\text{arch}} \times y_{\text{calc}}$ matrix. Rows are Archidekt labels; columns are calculator labels.

$y_{\text{arch}} \setminus y_{\text{calc}}$	2	3	4
2	3,042	3,459	462
3	2,307	11,727	4,617
4	384	2,991	7,416

Tables 17–20 report the four highlighted model matrices. Rows are $\hat{y}_{\text{arch}}$; columns are y_{calc}. The diagonal is exact agreement with the calculator.

Table 17: `df_random_forest`: highest exact agreement with y_{calc}.

$\hat{y}_{\text{arch}} \setminus y_{\text{calc}}$	2	3	4
2	3,518	2,375	102
3	2,142	14,829	5,523
4	73	973	6,870

I Interpretability Details

Table 21 reports directional summaries for the top DF features. These are conditional means on the interpretability validation split. They are descriptive and do not imply causality.

For the BC model, divergence lift separates concordant and divergent OOF rows. Cards enriched in concordant rows include *Blood Moon* (lift 35.707), *Winter Moon* (21.770), *Harbinger of the Seas* (10.914), *Ruination* (6.466), *Back to Basics* (6.038), *Imperial Seal* (5.897), *Chrome Mox* (5.297), and *Force of Will* (5.019). Cards enriched in sub-predicted rows include *Vorinclex, Voice of Hunger* (42.282), *Expropriate* (18.267), *Time Stretch* (15.885), and *Time Sieve* (15.290).

Table 18: `bc_decision_tree`: lowest exact agreement with y_{calc}.

$\hat{y}_{\text{arch}} \setminus y_{\text{calc}}$	2	3	4
2	3,516	4,488	820
3	1,811	9,511	4,976
4	406	4,178	6,699

Table 19: `df_gradient_boosting`: largest absolute gap between y_{arch} performance and calculator agreement.

$\hat{y}_{\text{arch}} \setminus y_{\text{calc}}$	2	3	4
2	4,511	4,794	524
3	1,061	11,483	4,668
4	161	1,900	7,303

J Reproducibility and AI Use

All experiments use fixed seeds and stored fold definitions. The nested-CV artifacts are stored under `experiments/model_id/`; voting artifacts are stored under `experiments/voting/`; selected models are recorded in `experiments/best_models.json`; interpretability artifacts are stored under `experiments/phase_j_interpretability/`. The processed data snapshot is distributed separately so that y_{calc} remains fixed to the collection-time values.

The project used AI assistance during planning, implementation review, report drafting, and article editing. All reported numbers in the article come from saved experiment artifacts or generated reports, and the final responsibility for data collection, code execution, result validation, and paper content remains with the authors.

Table 20: `bc_naive_bayes`: smallest absolute gap between y_{arch} performance and calculator agreement.

$\hat{y}_{\text{arch}} \setminus y_{\text{calc}}$	2	3	4
2	3,508	4,435	472
3	1,645	9,432	4,775
4	580	4,310	7,248

Table 21: DF feature means by predicted bracket for `df_gradient_boosting`.

Feature	Overall	Pred. 2	Pred. 3	Pred. 4
`game_changer_count`	1.737	0.001	1.499	4.146
`mass_land_denial_count`	0.052	0.000	0.000	0.207
`unique_atomic_combo_refs_count`	17.252	10.444	15.965	27.356
`tutor_count`	2.315	1.245	2.249	3.650
`cmc_1_count`	8.748	7.920	8.579	9.998